GUIDE DU PROJET · BUSI SPARK STREAMING

Comment le projet fonctionne, étape par étape

Ce guide explique tout le projet simplement : ce qu'il fait, chaque composant, comment ils communiquent, et les mots techniques utilisés. Il n'est pas nécessaire de connaître Spark ou le machine learning pour le lire.

---

1. L'idée en une phrase
2. Les données : la base BUSI
3. Vue d'ensemble
4. Le producer
5. Le consumer
6. Les caractéristiques
7. Le service ML
8. Le modèle ONNX
9. Le dashboard
10. Les fichiers échangés
11. Lancer le projet
12. Résultats
13. Glossaire

1 L'idée en une phrase

Le projet reçoit des échographies mammaires **au fil de l'eau**, comme si elles arrivaient d'un hôpital, et prédit pour chacune si elle montre une lésion **bénigne**, une lésion **maligne** (un cancer) ou un sein **normal**. Le modèle qui fait la prédiction s'améliore tout seul à mesure que de nouvelles images arrivent.

Le projet a deux objectifs à la fois :

- **Big data** : traiter un flux continu de données avec **Apache Spark**, un outil fait pour traiter de gros volumes de données en parallèle.
- **Machine learning** : apprendre à un programme à reconnaître les trois classes à partir d'exemples.

2 Les données : la base BUSI

BUSI (*Breast Ultrasound Images*) est une base publique d'échographies mammaires, collectée en 2018 auprès de 600 patientes. Chaque image est déjà classée par des médecins : c'est ce qui permet d'apprendre au modèle.

CLASSE	SIGNIFICATION	IMAGES
benign	Lésion non cancéreuse (kyste, fibroadénome…)	437
malignant	Tumeur cancéreuse	210
normal	Pas de lésion	133
Total		**780**

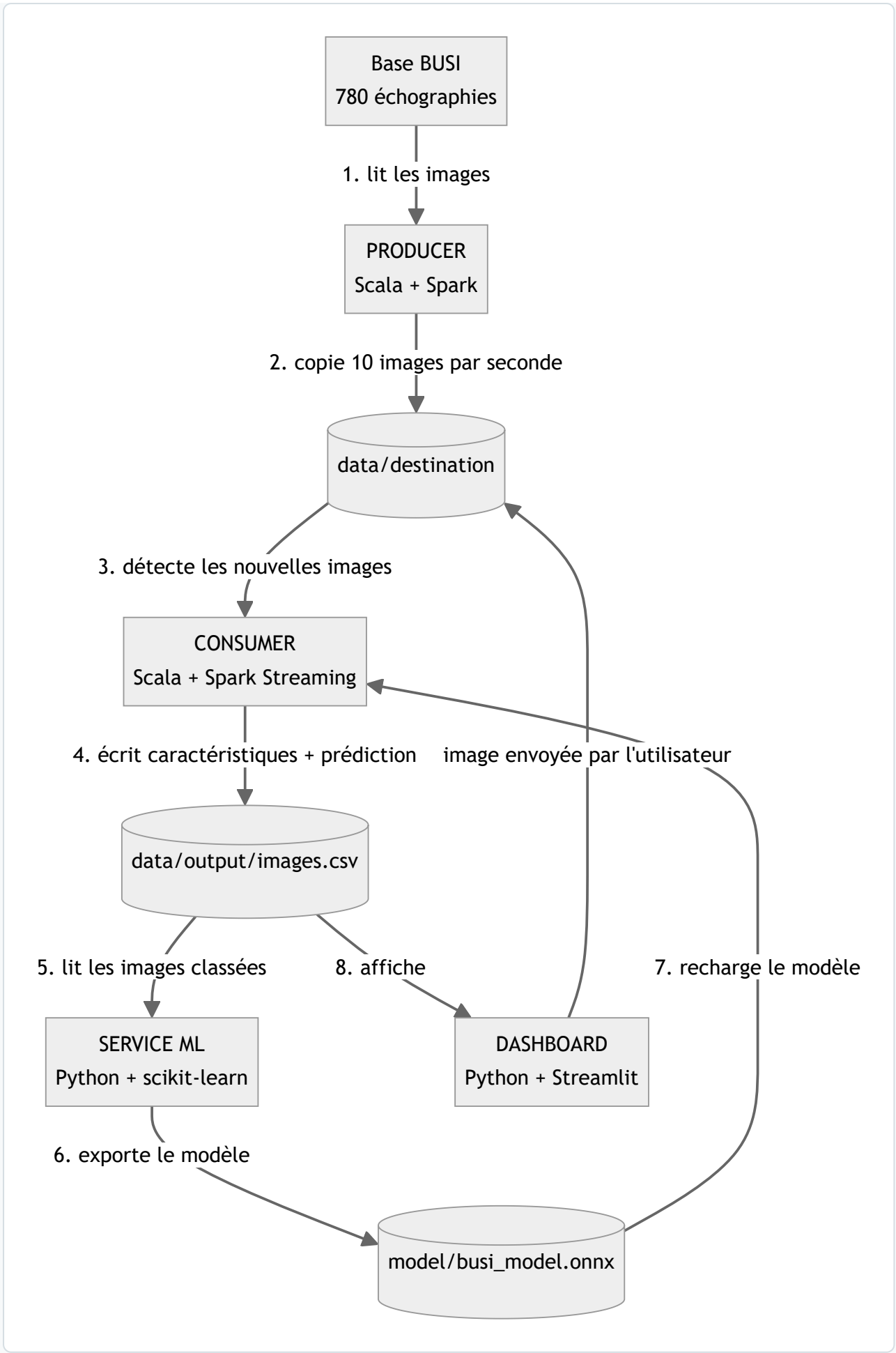
Chaque échographie est accompagnée d'un **masque** : une image noir et blanc où la lésion est coloriée en blanc. Le projet ne s'en sert pas, car une nouvelle échographie envoyée par un utilisateur n'aurait pas de masque.

La classe de chaque image est écrite dans son nom de fichier : `malignant (12).png` est une image maligne. Le projet lit la classe directement dans ce nom.

La base est rangée sur le Bureau, **à côté** du projet (`../Dataset_BUSI_with_GT`), et pas dedans : elle est trop lourde pour GitHub. Le dossier `data/Source` contient 27 images d'exemple pour un essai rapide.

3 Vue d'ensemble

Le projet est fait de **quatre programmes** qui tournent en même temps. Ils ne s'appellent jamais directement : ils communiquent en **déposant et en lisant des fichiers** dans des dossiers partagés.


Base BUSI
780 échographies
1. lit les images
PRODUCER
Scala + Spark
2. copie 10 images par seconde
data/destination
3. détecte les nouvelles images
CONSUMER
Scala + Spark Streaming
4. écrit caractéristiques + prédiction
image envoyée par l'utilisateur
data/output/images.csv
5. lit les images classées
8. affiche
7. recharge le modèle
SERVICE ML
Python + scikit-learn
DASHBOARD
Python + Streamlit
6. exporte le modèle
model/busi_model.onnx

> **Image simple :** c'est une chaîne de montage. Le producer pose des pièces sur un tapis roulant (le dossier `destination`). Le consumer mesure chaque pièce et note ses mesures dans un registre (`images.csv`). Le service ML étudie le registre pour apprendre à reconnaître les pièces, et donne ses conseils au consumer sous forme de fichier (le modèle). Le dashboard est l'écran de contrôle de l'atelier.

Les numéros 1 à 8 du schéma reviennent dans tout le guide et dans les commentaires du code.

4 Le producer : simuler l'arrivée des images

RÔLE	Faire comme si des échographies arrivaient en continu (étapes 1 et 2).
LANGAGE	Scala, avec **Spark Core** (la partie de base de Spark).
FICHIER	`src/main/scala/Producer/FileProducer.scala`

Dans un vrai hôpital, les images arriveraient une par une, au rythme des examens. Ici, on a déjà toutes les images : le producer les **distribue petit à petit** pour recréer cet effet de flux.

1. Il liste toutes les images `.png` de la base, sauf les masques (`_mask`).
2. Il les mélange au hasard, pour que les classes arrivent dans le désordre comme dans la réalité. Sans ça, les 437 benign arriveraient d'abord et le modèle ne verrait les normal qu'à la fin.
3. Il les découpe en lots de 10 images.
4. Pour chaque lot, il copie les images dans `data/destination`, puis attend 1 seconde avant le lot suivant.

Le détail important : la copie en deux temps

Copier une image prend un peu de temps. Si le consumer lisait une image en cours de copie, il trouverait un fichier à moitié écrit. Le producer copie donc d'abord l'image dans un dossier tampon (`data/staging`), que personne ne surveille, puis la **renomme** vers `data/destination`. Un renommage est instantané : l'image apparaît d'un coup, complète.

Les copies d'un lot sont réparties entre les cœurs du processeur grâce à un **RDD** Spark (voir le glossaire) : c'est là que le producer utilise Spark.

Tous les réglages (dossiers, 10 images par lot, 1 seconde de pause, mélange) sont dans `src/main/resources/application.conf`, section `producer`.

5 Le consumer : le cœur du projet

RÔLE	Traiter chaque image dès qu'elle arrive : la mesurer, la prédire, écrire les résultats (étapes 3, 4 et 7).
LANGAGE	Scala, avec **Spark Structured Streaming** (la partie de Spark qui traite des flux).
FICHIERS	`src/main/scala/Consumer/ImageConsumer.scala` et `OnnxModel.scala`

Comment il surveille le dossier : les micro-batchs

Spark ne traite pas les images une par une à l'instant exact où elles arrivent. **Toutes les 5 secondes**, il regarde le dossier `data/destination`, repère les fichiers qu'il n'a pas encore vus, et les traite ensemble (au plus 20 à la fois). Ce petit paquet s'appelle un **micro-batch**.

> **Image simple :** un facteur qui passe relever la boîte aux lettres toutes les 5 minutes, plutôt que de guetter chaque lettre.

Ce qu'il fait de chaque image

1. Il lit la classe dans le nom du fichier (`benign`, `malignant` ou `normal`). Pour une image envoyée depuis le dashboard (`upload_…png`), la classe reste vide : on ne la connaît pas.
2. Il décode l'image, la convertit en niveaux de gris et calcule ses **caractéristiques** : 39 nombres qui la résument (voir la section suivante).
3. Si un modèle existe, il lui donne ces nombres et obtient une prédiction, par exemple « malignant à 81 % ».
4. Il ajoute une ligne par image dans `data/output/images.csv`.

Trois calculs en parallèle sur le même flux

REQUÊTE	CE QU'ELLE PRODUIT	FICHIER
`images`	Une ligne par image : caractéristiques + prédiction	`images.csv`
`stats_by_label`	Moyennes par classe depuis le début (taille, luminosité, contraste…)	`stats_by_label.csv`
`window_counts`	Nombre d'images par classe, par tranche de 10 secondes	`window_counts.csv`

Deux notions de streaming à connaître

- **Checkpoint** (`data/checkpoint`) : la mémoire de Spark. Il y note les fichiers déjà traités et les totaux en cours. Si on arrête et relance le consumer, il reprend où il s'était arrêté au lieu de tout retraiter. C'est pour ça qu'il faut vider ce dossier pour « repartir de zéro ».
- **Watermark** (30 secondes) : le retard maximal accepté pour une image. Pour compter les images par tranche de 10 secondes, Spark doit savoir quand une tranche est terminée : passé 30 secondes, il considère qu'aucune image en retard n'arrivera plus pour cette tranche et peut l'oublier.

Le rechargement du modèle

À chaque micro-batch, le consumer vérifie si le fichier du modèle a changé. Si le service ML en a exporté un nouveau, il le charge **sans s'arrêter** : les images suivantes sont prédites avec le nouveau modèle. Chaque ligne de `images.csv` garde la version du modèle qui l'a prédite.

6 Les caractéristiques : décrire une image avec des nombres

Le modèle **ne voit pas l'image**. Il reçoit une liste de nombres qui la résument, calculés par le consumer. C'est comme décrire une personne par sa taille, son poids et la couleur de ses yeux plutôt que par sa photo : le modèle ne sera bon que si ces nombres capturent ce qui distingue les classes.

Une image en niveaux de gris est un tableau de pixels, chacun avec une valeur de 0 (noir) à 255 (blanc). Une échographie BUSI fait environ 500 × 500 pixels, soit 250 000 valeurs. Le consumer les résume en 39 nombres, dont 37 servent au modèle :

GROUPE	NOMBRES	CE QUE ÇA MESURE, SIMPLEMENT
Intensité	4	**Moyenne** : l'image est-elle claire ou sombre ? **Écart-type** : est-elle contrastée ? **Entropie** : a-t-elle beaucoup de nuances différentes ? **Gradient** : y a-t-il beaucoup de contours ?
Histogramme	16	Les 256 niveaux de gris regroupés en 16 tranches : quelle part de l'image est très sombre, un peu sombre… très claire.
Texture GLCM	4	Deux pixels voisins se ressemblent-ils ? Mesure si la zone est lisse ou granuleuse, régulière ou désordonnée.
Texture LBP	10	Pour chaque pixel, on regarde si ses 8 voisins sont plus clairs ou plus sombres. Ça donne un « motif » : bord, coin, zone plate ou motif irrégulier. On compte la part de chaque motif.
Zone centrale	3	Luminosité et contraste du centre de l'image, où la lésion se trouve souvent, et comparaison avec l'image entière.
Taille (non utilisée)	2	Largeur et hauteur. Affichées, mais retirées du modèle : dans BUSI, la taille dépend de la classe et le modèle apprendrait ce biais au lieu de regarder l'échographie.

Pourquoi la texture compte : une tumeur maligne a souvent des **contours irréguliers** et un intérieur **hétérogène**, alors qu'un kyste bénin est plutôt rond, lisse et sombre de façon uniforme. Les mesures LBP et GLCM capturent justement ces différences.

Voici un extrait de `images.csv` : l'image `malignant (1).png` envoyée comme une image inconnue (classe vide), puis prédite par le modèle :

```
fileName,label,meanIntensity,stdIntensity,glcmContrast,lbp9,centerRatio,predict
"upload_test_malignant_1.png",,117.97,61.45,0.44235,0.08066,0.6957,"malignant",
```

Son `centerRatio` de 0,70 signifie que le centre de l'image est 30 % plus sombre que l'ensemble : la lésion sombre au centre se voit dans ce seul nombre.

7 Le service ML : apprendre à reconnaître les classes

RÔLE	Entraîner le modèle à partir des images déjà classées, et l'exporter (étapes 5 et 6).
LANGAGE	Python, avec **scikit-learn** (bibliothèque de machine learning).
FICHIER	`ml_service/train_model.py`

Le modèle : une forêt aléatoire

Un **arbre de décision** est une suite de questions oui/non sur les caractéristiques : « le centre est-il plus sombre que le reste ? », « y a-t-il beaucoup de motifs irréguliers ? »… jusqu'à une réponse (benign, malignant ou normal). L'arbre apprend tout seul quelles questions poser, à partir des exemples.

Une **forêt aléatoire**, c'est 200 arbres, chacun entraîné sur un tirage différent des images. Pour une nouvelle image, les 200 arbres votent. La classe qui a le plus de voix gagne, et la part des voix donne la **confiance** : environ 163 arbres sur 200 pour « malignant » donnent 81,5 %.

> **Image simple :** demander l'avis de 200 médecins qui ont chacun vu des patientes un peu différentes, puis suivre la majorité. Un médecin seul peut se tromper ; la majorité se trompe moins.

Comment il tourne

1. Toutes les 10 secondes, il relit `images.csv` et garde les images dont la classe est connue.
2. Au premier passage avec assez d'images, puis chaque fois qu'assez de nouvelles images sont arrivées, il entraîne un nouveau modèle.
3. Il évalue le modèle (voir ci-dessous), l'exporte en ONNX dans `model/busi_model.onnx`, et écrit ses scores dans `data/output/model_metrics.json`.
4. Il s'arrête tout seul après 10 minutes sans nouvelle image, avec un dernier entraînement s'il reste des images non utilisées.

Comment on sait si le modèle est bon : la validation croisée

Tester un modèle sur les images qu'il a apprises serait tricher : il les connaît déjà. La **validation croisée** coupe les images en 5 groupes. On entraîne sur 4 groupes, on teste sur le 5e, et on recommence 5 fois en changeant le groupe de test. Au final, chaque image a été prédite par un modèle qui ne l'avait jamais vue.

On obtient deux résultats :

- la **précision** : la part des images bien classées (74,5 %) ;
- la **matrice de confusion** : un tableau qui montre, pour chaque vraie classe, ce que le modèle a répondu. On y voit par exemple combien de cancers ont été pris pour des lésions bénignes.

Les options utiles

OPTION	EFFET
`--once`	Un seul entraînement, puis arrêt
`--retrain-every 50`	Ré-entraîner toutes les 50 nouvelles images
`--balance`	Même nombre d'images par classe (133 chacune)
`--idle-timeout 600`	Arrêt après 600 secondes sans nouvelle image

8 Le modèle ONNX : le pont entre Python et Scala

Le modèle est entraîné en **Python**, mais utilisé par le consumer en **Scala**. Les deux langages ne peuvent pas s'échanger directement un objet scikit-learn. **ONNX** (*Open Neural Network Exchange*) est un format de fichier standard pour les modèles : Python l'écrit, et la bibliothèque **ONNX Runtime** le lit et l'exécute en Scala.

> **Image simple :** un PDF. On l'écrit avec Word, on le lit avec n'importe quel lecteur, sans avoir besoin de Word.

Le fichier contient aussi des **métadonnées** : la liste des caractéristiques dans l'ordre attendu, le nom des classes et la version du modèle. Le consumer n'a donc rien à deviner : si on ajoute une caractéristique côté Python, il s'adapte tout seul.

Avant d'exporter, le service ML vérifie que le modèle ONNX donne exactement les mêmes probabilités que le modèle Python. Sinon, l'export est annulé.

9 Le dashboard : voir ce qui se passe

RÔLE	Afficher les résultats en direct et permettre d'envoyer ses propres images (étape 8).
LANGAGE	Python, avec **Streamlit** (crée une page web à partir d'un script Python).
FICHIER	`dashboard/app.py`, ouvert sur `http://localhost:8501`

Le dashboard **ne calcule rien** : toutes les 3 secondes, il relit les fichiers de `data/output` et les affiche.

- **Barre latérale** : état du pipeline (actif ou en attente), nombre d'images reçues par classe, précision et version du modèle.
- **Images par classe** : images reçues et images utilisées pour l'entraînement.

- **Erreurs du modèle** : la matrice de confusion.
- **Dernières images reçues** : une galerie avec la vraie classe, la prédiction et ✅ ou ❌.
- **Prédire mes images** : on dépose une échographie, le dashboard la place dans `data/destination` sous le nom `upload_…png`, et le consumer la prédit au micro-batch suivant. Comme sa classe est inconnue, elle ne sert pas à l'entraînement.

10 Les fichiers échangés

Comme les programmes communiquent par fichiers, savoir qui écrit et qui lit quoi permet de comprendre tout le projet.

DOSSIER OU FICHIER	ÉCRIT PAR	LU PAR	CONTENU
`data/destination/`	Producer, dashboard	Consumer, dashboard	Les images du flux
`data/staging/`	Producer, dashboard	–	Tampon pendant la copie
`data/output/images.csv`	Consumer	Service ML, dashboard	Une ligne par image
`data/output/stats_by_label.csv`	Consumer	Dashboard	Moyennes par classe
`data/output/window_counts.csv`	Consumer	Dashboard	Images par tranche de 10 s
`data/output/model_metrics.json`	Service ML	Dashboard	Précision, matrice de confusion
`data/output/model_history.csv`	Service ML	–	Une ligne par entraînement
`model/busi_model.onnx`	Service ML	Consumer	Le modèle
`data/checkpoint/`	Consumer (Spark)	Consumer (Spark)	Mémoire du streaming

Pour éviter qu'un programme lise un fichier à moitié écrit, chaque écriture importante passe par un fichier temporaire, ensuite renommé d'un coup.

11 Lancer le projet

Il faut Java 17, sbt (l'outil qui compile et lance le code Scala), Python 3.11 avec `pip install -r requirements.txt`, et sous Windows les fichiers Hadoop `winutils.exe` et `hadoop.dll`. Ensuite, dans quatre terminaux, depuis le dossier du projet :

```
sbt "runMain Consumer.ImageConsumer"                  # 1. consumer
python ml_service/train_model.py --retrain-every 50   # 2. service ML
streamlit run dashboard/app.py                        # 3. dashboard
sbt "runMain Producer.FileProducer"                   # 4. producer
```

L'ordre compte : le consumer doit être prêt avant que le producer envoie des images. Pour tout recommencer, on vide `data/destination`, `data/checkpoint`, `data/output` et `data/staging`.

12 Résultats

Sur les 780 images, en validation croisée, le modèle classe correctement **74,5 %** des images.

CLASSE	BIEN RECONNUES	PART
benign	356 / 437	81 %
malignant	142 / 210	68 %
normal	83 / 133	62 %

Pour comparaison, répondre toujours « benign » donnerait 56 %. Le chiffre le plus important dans un contexte médical est la part de cancers détectés (68 %) : rater un cancer est bien plus grave qu'une fausse alerte. Le détail des choix qui ont mené à ce résultat est dans le rapport technique séparé.

13 Glossaire

Apache Spark

Outil pour traiter de grandes quantités de données en répartissant le travail sur plusieurs cœurs ou plusieurs machines. Ici, il tourne sur un seul PC (`local[*]` : tous les cœurs du processeur).

Scala

Langage dans lequel Spark est écrit ; il tourne sur la machine virtuelle Java.

RDD

Resilient Distributed Dataset : une liste de données que Spark découpe en morceaux et traite en parallèle. Utilisé par le producer.

DataFrame

Un tableau de données (lignes et colonnes) manipulé par Spark, comme une feuille Excel. Utilisé par le consumer.

Structured Streaming

La partie de Spark qui traite un flux de données qui ne s'arrête jamais, comme s'il s'agissait d'un tableau qui grandit.

Micro-batch

Le petit paquet de nouvelles données que Spark traite à chaque passage (ici, toutes les 5 secondes).

UDF

User Defined Function : une fonction écrite par nous et appliquée par Spark à chaque ligne. Ici, le calcul des caractéristiques d'une image.

Checkpoint

Le dossier où Spark enregistre où il en est, pour reprendre après un arrêt.

Watermark

Le retard maximal accepté pour une donnée qui arrive en retard (ici 30 secondes).

Caractéristique

Un nombre qui décrit un aspect de l'image (luminosité, texture…). En anglais : *feature*.

Entraînement

La phase où le modèle apprend à partir d'exemples dont on connaît la bonne réponse.

Forêt aléatoire

Un modèle fait de nombreux arbres de décision qui votent.

Validation croisée

Méthode d'évaluation où chaque image est testée par un modèle qui ne l'a pas apprise.

Précision

La part des images bien classées, toutes classes confondues.

Rappel

Pour une classe donnée, la part de ses images que le modèle a retrouvées. Le rappel malignant est la part des cancers détectés.

Matrice de confusion

Tableau croisant la vraie classe et la classe prédite : la diagonale montre les bonnes réponses, le reste les erreurs.

Biais

Un raccourci que le modèle apprend parce qu'il existe par hasard dans les données (ici, la taille des images), mais qui n'a rien à voir avec ce qu'on veut reconnaître.

ONNX

Format de fichier standard pour échanger un modèle entre langages (ici, de Python vers Scala).

Streamlit

Bibliothèque Python qui transforme un script en page web interactive.

Code source : github.com/Ikraam26/BUSI_Spark_Streaming. Les chiffres viennent du modèle entraîné le 16 octobre 2026.